Documentation

Overview

This project implements a secure social bookshelf application, where users can create and share private reading lists (shelves) and post reviews of book. The book selections and reviews are only readable by members of that shelf. To all other users of the system, the content appears as ciphertext.

Uses a Flask REST API, SQLite database, and the Python `cryptography` library.

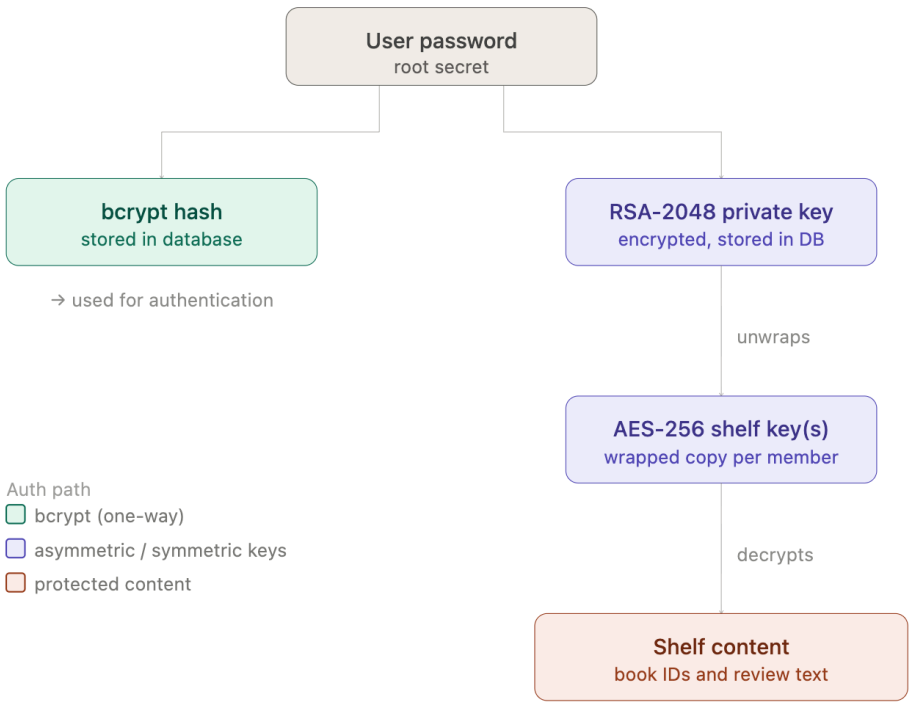
This project is based upon the https://openlibrary.org/ website and API. It is an ebook library website which allows users to borrow books online.

Protocol Design

Cryptographic Primitives

Primitive	Algorithm	Parameters
Symmetric encryption	AES-GCM	256-bit key, 96-bit nonce
Asymmetric encryption	RSA-OAEP	2048-bit key, SHA-256
Key wrapping (OAEP)	MGF1	SHA-256
Certificate format	X.509 v3	Self-signed, 365-day validity
Password hashing	bcrypt	Cost factor 12
Private key storage	PKCS#8	BestAvailableEncryption (password-derived)

Keys




```
import datetime

from cryptography import x509
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import padding, rsa
from cryptography.x509.oid import NameOID

from app.crypto import generate_aes_key

RSA_KEY_SIZE = 2048
```

```
RSA_PUBLIC_EXPONENT = 65537
EXPIRY = 365


def generate_rsa_keypair() -> tuple:
    private_key = rsa.generate_private_key(
        public_exponent=RSA_PUBLIC_EXPONENT,
        key_size=RSA_KEY_SIZE,
    )
    return private_key, private_key.public_key()


def serialize_private_key(private_key, password: bytes) -> bytes:
    return private_key.private_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PrivateFormat.PKCS8,
        encryption_algorithm=serialization.BestAvailableEncryption(password),
    )

def deserialize_private_key(pem_data: bytes, password: bytes):
    return serialization.load_pem_private_key(pem_data, password=password)

def wrap_group_key(aes_key: bytes, recipient_public_key) -> bytes:
    return recipient_public_key.encrypt(
        aes_key,
        padding.OAEP(
            mgf=padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None,
        ),
    )


def unwrap_group_key(wrapped_key: bytes, private_key) -> bytes:
    try:
        return private_key.decrypt(
            wrapped_key,
            padding.OAEP(
                mgf=padding.MGF1(algorithm=hashes.SHA256()),
                algorithm=hashes.SHA256(),
                label=None,
            ),
        )
    except Exception as e:
        raise ValueError(f"Failed to unwrap group key: {e}")
```

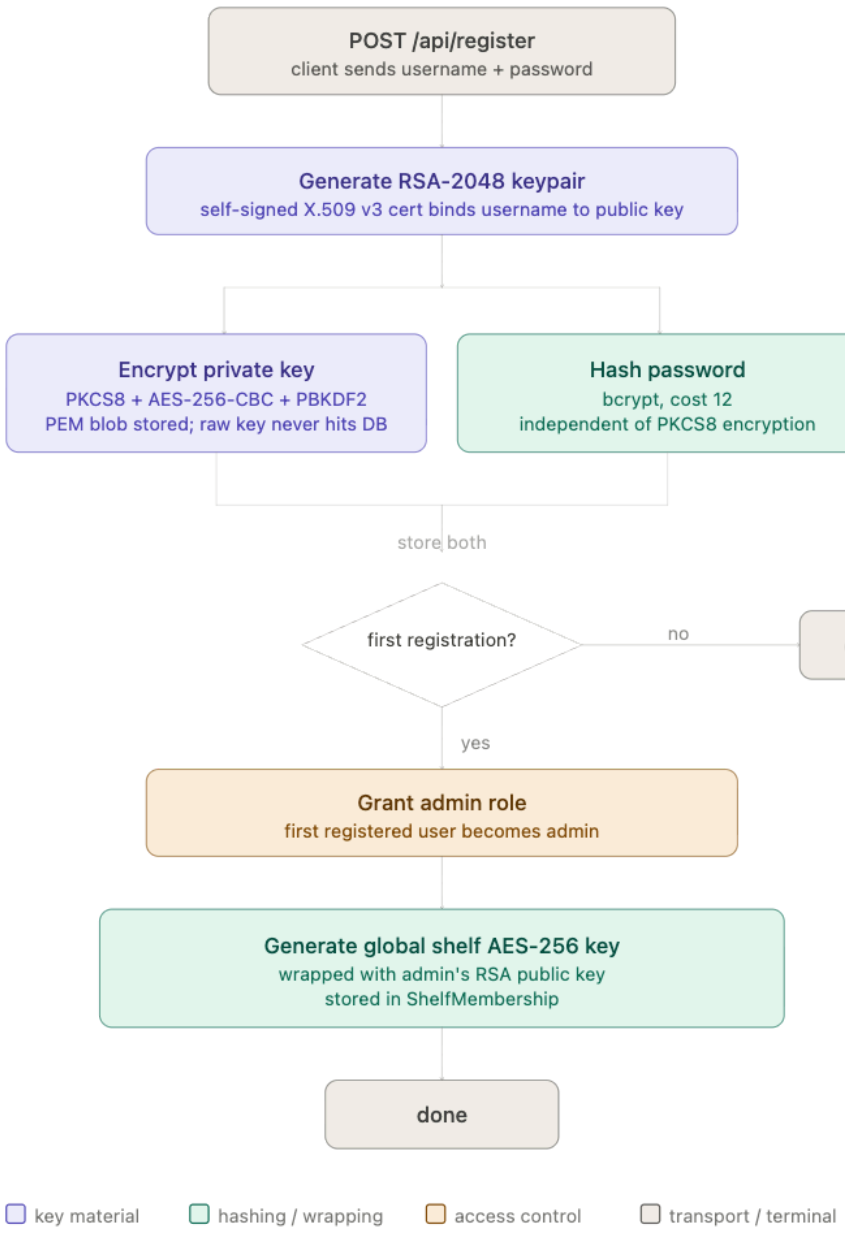
Each shelf has its own independent AES-256-GCM key. Each member of a shelf holds a copy of that key, encrypted with their own RSA-2048 public key. Only the RSA private keyholder can recover the AES shelf key. The RSA private key itself is only recoverable with the user's password.

SHA-256 is used twice as the two hashes serve different roles. The `algorithm=hashes.SHA256()` is the OAEP hash, used to hash the optional label and define the length of the seed mask. Meanwhile, `mgf=padding.MGF1(algorithm=hashes.SHA256())` is the hash used inside the MGF1 (Mask Generation Function 1). which creates a mask to block the data and ensure the plaintext produces a different ciphertext each time.

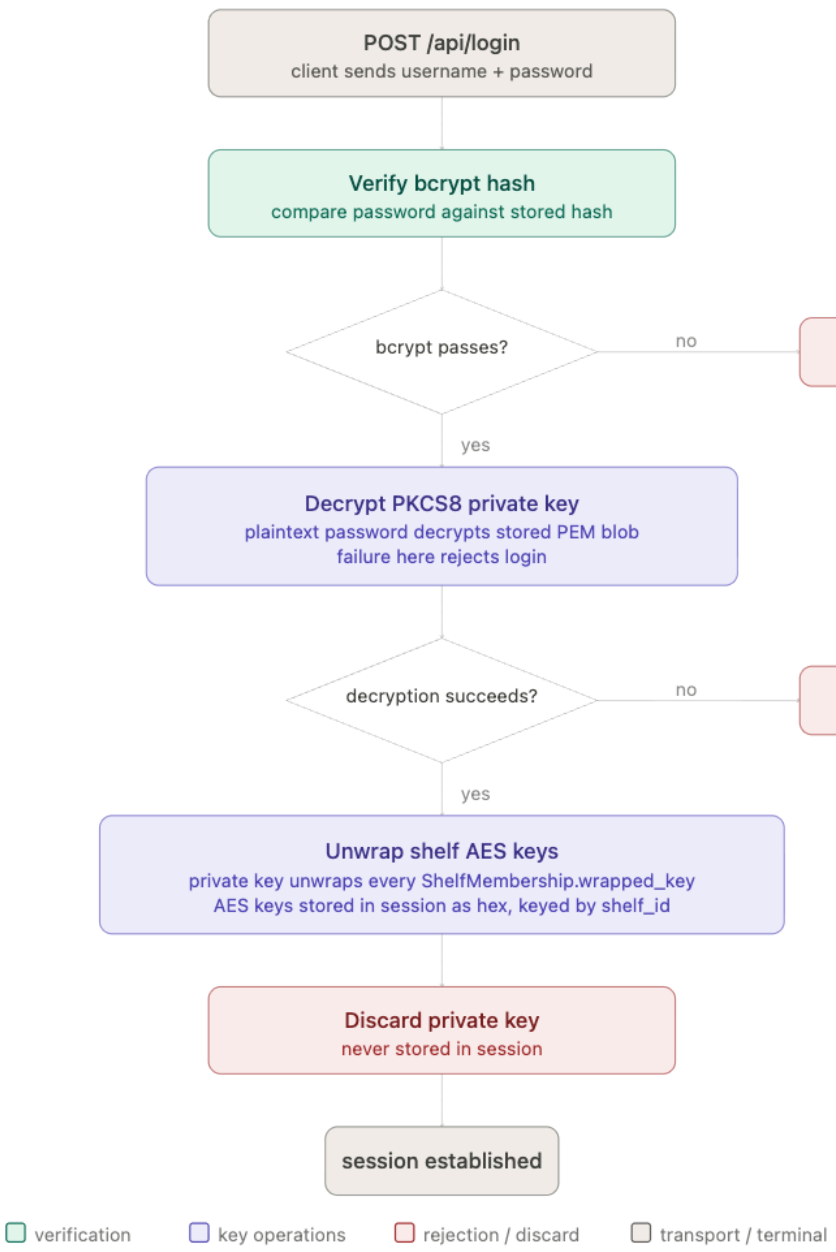
Coincidentally, both happen to be the standard and recommended configuration of SHA-256.

Authentication and Session Flow

Registration



Login

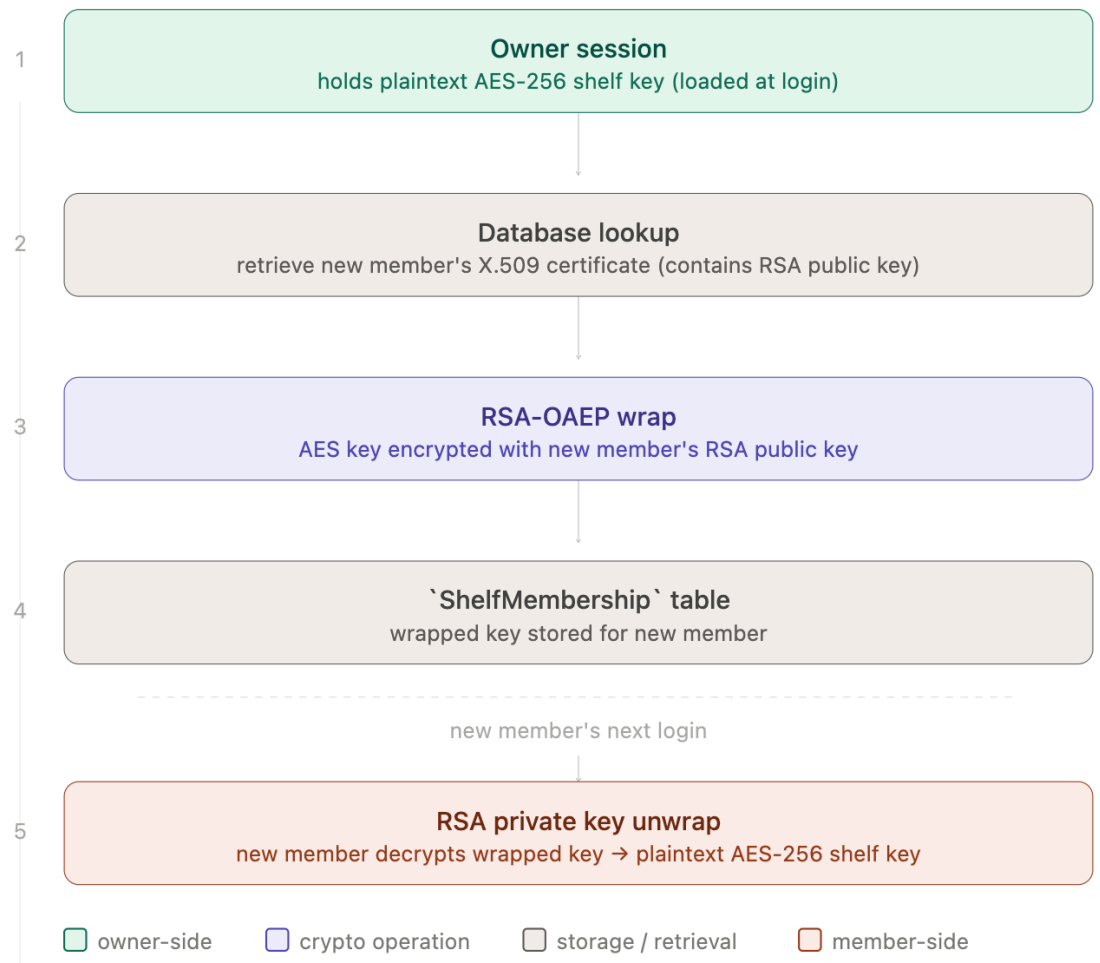


Group Membership and Key Management

Adding a member

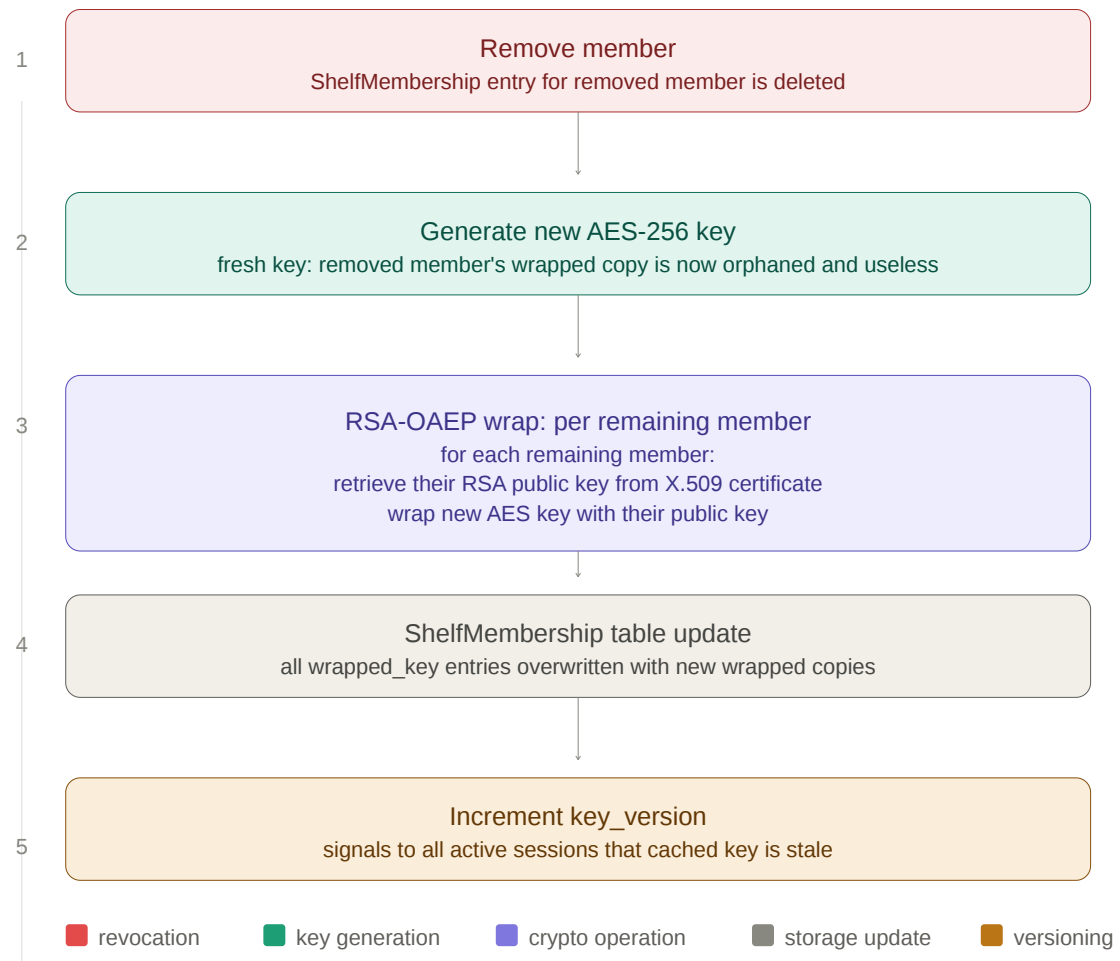
```
def add_member(aes_key: bytes, new_member_cert_pem: bytes) -> bytes:
    cert = deserialize_certificate(pem_data=new_member_cert_pem)
    return wrap_group_key(aes_key, cert.public_key())


def remove_member(remaining_member_certs_pem: list) -> tuple:
    aes_key = generate_aes_key()
    new_group = {}
    for cert_pem in remaining_member_certs_pem:
        cert = deserialize_certificate(pem_data=cert_pem)
        username = get_username_from_cert(cert)
        new_group[username] = wrap_group_key(aes_key, cert.public_key())
    return aes_key, new_group
```

When a shelf owner adds a new member, the following occurs



Member Removal

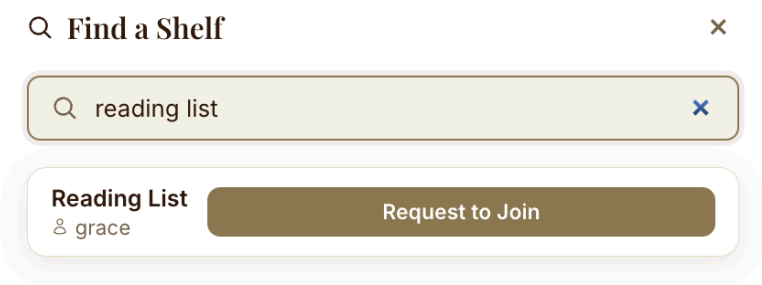
Member removal results in a full re-keying of the shelf, as follows:



This is to make sure the removed member's copy of the old key cannot decrypt further content so that historical ciphertext encrypted under the old key remains in the database but no longer decryptable by the removed user.

Join Request

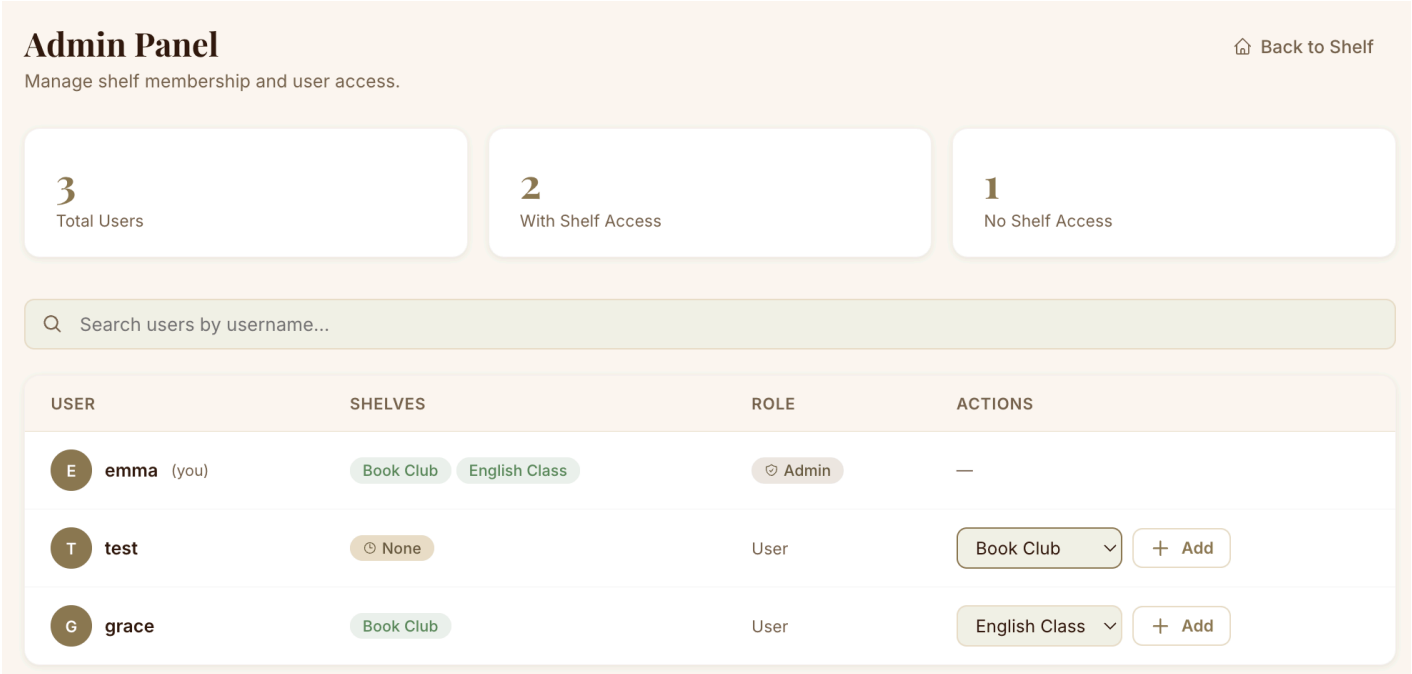
If a user wants to join a shelf, they can search it up and request to join. The user posts a shelf access request without a key. The owner approves the request. The key is wrapped from the owner when they approve.



User finds shelves

Invitation

Meanwhile, for invitation, the wrapped key is pre-computed at invite time. If the invitee accepts, they give their password to get the private key which can be used to unwrap the stored private key for access to the shelf.



Admin panel for shelf owner

Public-Key Certificates

```
def generate_certificate(username: str, private_key, public_key) -> x509.Certificate:
    subject = issuer = x509.Name([
        x509.NameAttribute(NameOID.COMMON_NAME, username),
        x509.NameAttribute(NameOID.ORGANIZATION_NAME, "Secure Bookshelf"),
    ])

    now = datetime.datetime.now(datetime.timezone.utc)
    cert = (
        x509.CertificateBuilder()
        .subject_name(subject)
        .issuer_name(issuer)
        .public_key(public_key)
        .serial_number(x509.random_serial_number())
        .not_valid_before(now)
        .not_valid_after(now + datetime.timedelta(days=EXPIRY))
        .sign(private_key, hashes.SHA256())
    )
    return cert


def serialize_certificate(cert: x509.Certificate) -> bytes:
    return cert.public_bytes(serialization.Encoding.PEM)
```

```
def deserialize_certificate(pem_data: bytes) -> x509.Certificate:
    return x509.load_pem_x509_certificate(pem_data)


def get_username_from_cert(cert: x509.Certificate) -> str:
    return cert.subject.get_attributes_for_oid(NameOID.COMMON_NAME)[0].value
```

For each user, a self-signed X.509 v3 certificate is issued when they register. This binds the person's username to their RSA-2048 public key. This allows the user to bind their public key to the private shelf key. The certificate also ensures the system can associate the key with the user. For this application, the certificates are self-signed.

Content Encryption

```
import os
import base64
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

AES_KEY_SIZE = 32
NONCE_SIZE = 12


def generate_aes_key() -> bytes:
    return os.urandom(AES_KEY_SIZE)


def encrypt_message(plaintext: str, aes_key: bytes) -> str:
    aesgcm = AESGCM(aes_key)
    nonce = os.urandom(NONCE_SIZE)
    ciphertext = aesgcm.encrypt(nonce, plaintext.encode("utf-8"), None)
    raw = nonce + ciphertext
    return base64.b64encode(raw).decode("utf-8")


def decrypt_message(b64_ciphertext: str, aes_key: bytes) -> str | None:
    try:
        raw = base64.b64decode(b64_ciphertext)
        nonce = raw[:NONCE_SIZE]
        ciphertext = raw[NONCE_SIZE:]
        aesgcm = AESGCM(aes_key)
        plaintext_bytes = aesgcm.decrypt(nonce, ciphertext, None)
        return plaintext_bytes.decode("utf-8")
    except Exception:
        return None


def is_encrypted(text: str) -> bool:
    if not text:
        return False
    try:
        raw = base64.b64decode(text, validate=True)
        return len(raw) > NONCE_SIZE
    except Exception:
        return False
```

The OpenLibrary identifier for each book is encrypted with the shelf's AES key. For users who are not a member of the shelf, the reviews are shown as ciphertext.

Each review is also encrypted with the shelf's AES key.

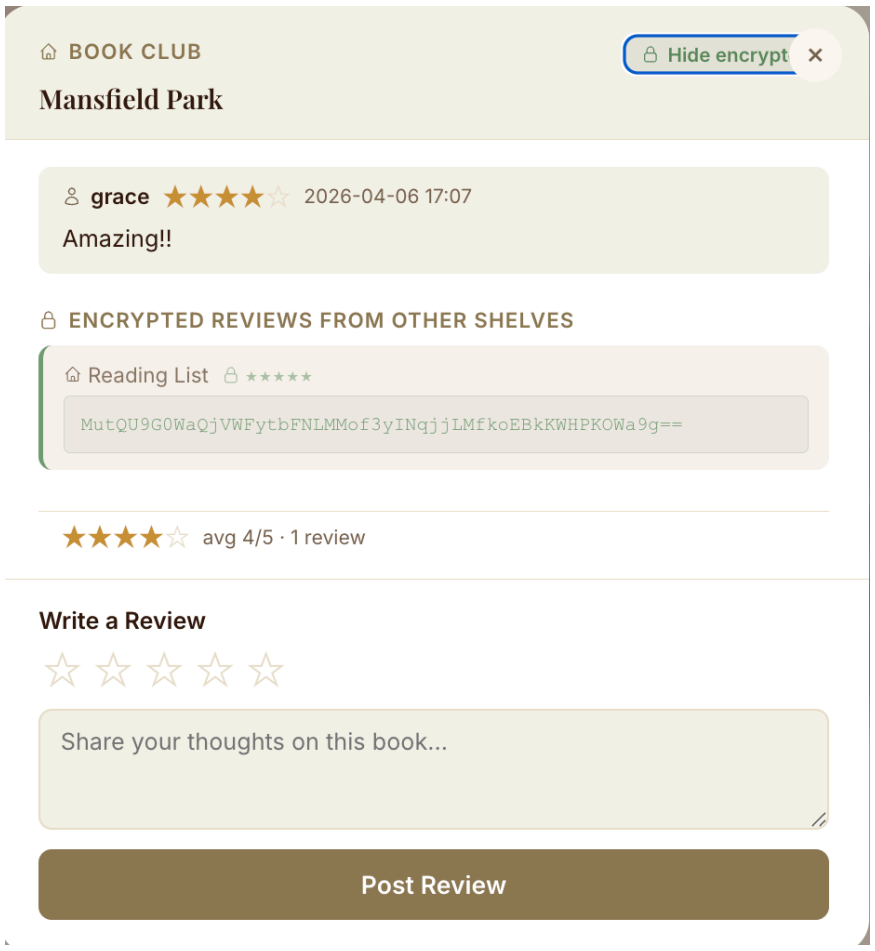
AES-GCM provides confidentiality and integrity. Confidentiality is because the content is unreadable without the key, and integrity because any tampering with the ciphertext is detected at decription via the authentication tag.

The 12-byte nonce is generated for each encryption and added to the ciphertext before the base64 encoding for storing. A new nonce with the same key is bad security as an attacker that sees both cipher texts can easily hack the plain texts and break the authentication tag. Generating random nonces per call makes this unlikely. We prepend the nonce as it doesn't need to be secret; only unique. This allows it to produce a single self-contained blob that can be stored in a database. Upon decryption, the first 12 bytes can be extracted to look at the nonce adn the rest passed on to decrypt. We used a 96 bit nonce as this is the NIST-recommended size for GCM.

To allow users to find a shelf, there is a SHA-256 hash of the plaintext work id stored along the ciphertext, which lets the user know that a book as reviews on the shelf they don't belong to. This allows non-member users to see that reviews exist without knowing the full shelf's book list.

We use base64-encode for the result as it can be printable as ASCII.

We check the length with is_encrypted as it's just a `nonce || ciphertext`. It should be 12 bytes for the nonce, 1 byte of ciphertext and 16 bytes of the auth tag. This is 29 bytes and we check if the length is longer than the nonce size to see if it's already encrypted.



Review posting and viewing

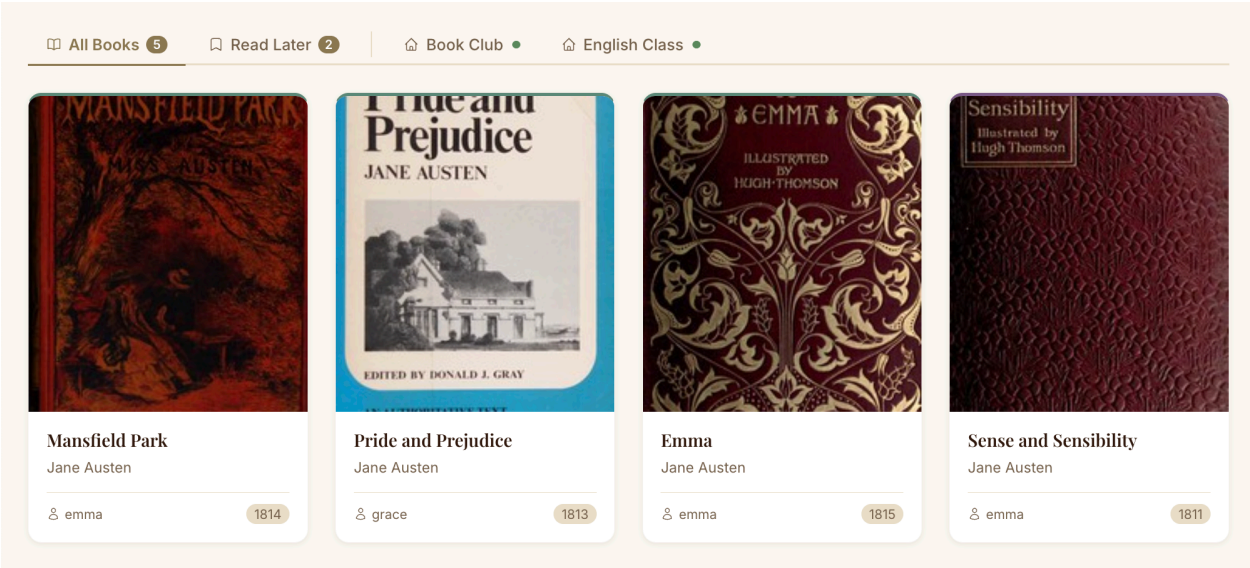
OpenLibrary API

```
import time
from concurrent.futures import ThreadPoolExecutor, as_completed

import requests

API_BASE = "https://openlibrary.org"
TIMEOUT = 5
CACHE_TTL = 3600

_book_cache: dict[str, tuple[dict | None, float]] = {}


def search_books(query: str, limit: int = 10) -> list[dict]:
    try:
        resp = requests.get(
            f"{API_BASE}/search.json",
            params={"q": query, "limit": limit, "fields": "key,title,author_name,first_publis
```

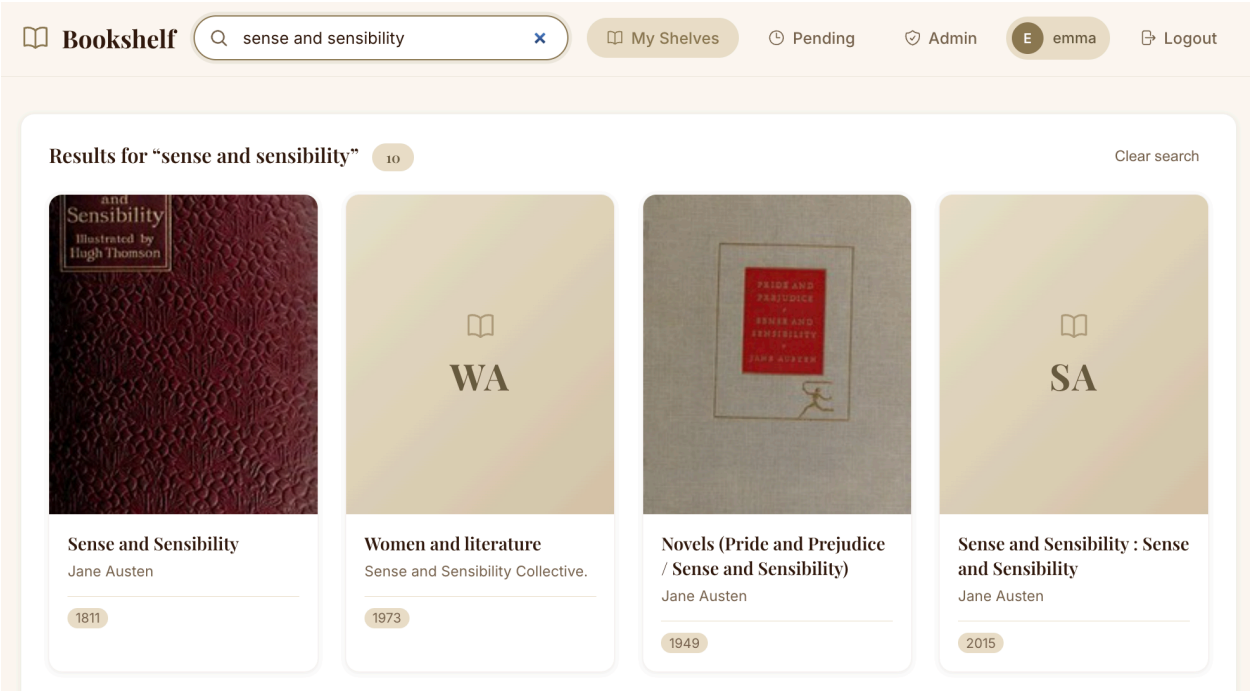
```
h_year,cover_i"},
            timeout=TIMEOUT,
        )
        if resp.status_code == 200:
            return [
                {
                    "work_id": doc["key"].replace("/works/", ""),
                    "title": doc.get("title", "Unknown"),
                    "author": ", ".join(doc.get("author_name", [])) or "Unknown",
                    "year": doc.get("first_publish_year"),
                    "cover_id": doc.get("cover_i"),
                }
                for doc in resp.json().get("docs", [])
                if "key" in doc
            ]
    except requests.RequestException:
        pass
    return []


def _fetch_book(work_id: str) -> dict | None:
    try:
        resp = requests.get(
            f"{API_BASE}/search.json",
            params={
                "q": f"key:/works/{work_id}",
                "fields": "key,title,author_name,first_publish_year,cover_i",
                "limit": 1,
            },
            timeout=TIMEOUT,
        )
        if resp.status_code == 200:
            docs = resp.json().get("docs", [])
            if docs:
                doc = docs[0]
                return {
                    "work_id": work_id,
                    "title": doc.get("title", "Unknown"),
                    "author": ", ".join(doc.get("author_name", [])) or None,
                    "year": doc.get("first_publish_year"),
                    "cover_id": doc.get("cover_i"),
                    "description": "",
                }
    except requests.RequestException:
        pass

    try:
        resp = requests.get(f"{API_BASE}/works/{work_id}.json", timeout=TIMEOUT)
        if resp.status_code == 200:
            data = resp.json()
            desc = data.get("description", "")
            if isinstance(desc, dict):
                desc = desc.get("value", "")
            covers = data.get("covers", [])
            cover_id = covers[0] if covers else None
            return {
                "work_id": work_id,
                "title": data.get("title", "Unknown"),
```

```
                "description": desc,
                "cover_id": cover_id,
                "author": None,
                "year": None,
            }
    except requests.RequestException:
        pass

    return None


def get_book(work_id: str) -> dict | None:
    now = time.time()
    if work_id in _book_cache:
        result, ts = _book_cache[work_id]
        if now - ts < CACHE_TTL:
            return result
    result = _fetch_book(work_id)
    _book_cache[work_id] = (result, now)
    return result


def get_books_batch(work_ids: list[str]) -> dict[str, dict]:
    result: dict[str, dict] = {}
    to_fetch: list[str] = []
    now = time.time()

    for wid in work_ids:
        if wid in _book_cache:
            cached, ts = _book_cache[wid]
            if now - ts < CACHE_TTL:
                if cached:
                    result[wid] = cached
                continue
        to_fetch.append(wid)

    if to_fetch:
        with ThreadPoolExecutor(max_workers=min(10, len(to_fetch))) as executor:
            futures = {executor.submit(_fetch_book, wid): wid for wid in to_fetch}
            for future in as_completed(futures):
                wid = futures[future]
                try:
                    book = future.result()
                except Exception:
                    book = None
                _book_cache[wid] = (book, now)
                if book:
                    result[wid] = book

    return result
```

API usage for the library website, https://openlibrary.org/. Supports searching, fetching and getting books.



Shelf display



Search Function

Database Schema

> For documentation brevity, I have removed parts of the database not directly related to the assignment. See the GitHub for more information.

```
class ShelfMembership(Base):
    __tablename__ = "shelf_memberships"
    __table_args__ = (UniqueConstraint("shelf_id", "username"),)

    id = Column(Integer, primary_key=True, autoincrement=True)
    shelf_id = Column(Integer, nullable=False)
    username = Column(String(64), nullable=False)
    wrapped_key = Column(LargeBinary, nullable=False)
    key_version = Column(Integer, default=1, nullable=False)
    updated_at = Column(DateTime, default=datetime.now, onupdate=datetime.now)

class ShelfAccessRequest(Base):
    __tablename__ = "shelf_access_requests"
    __table_args__ = (UniqueConstraint("shelf_id", "target_username"),)

    id = Column(Integer, primary_key=True, autoincrement=True)
    shelf_id = Column(Integer, nullable=False)
    target_username = Column(String(64), nullable=False)
    request_type = Column(String(16), nullable=False)
    wrapped_key = Column(LargeBinary, nullable=True)
    created_at = Column(DateTime, default=datetime.now)
```

```
class ShelfBook(Base):
    __tablename__ = "shelf_books"

    id = Column(Integer, primary_key=True, autoincrement=True)
    shelf_id = Column(Integer, nullable=False)
    work_id_enc = Column(Text, nullable=False)
    work_id_hash = Column(String(64), nullable=True)
    added_by = Column(String(64), nullable=False)
    created_at = Column(DateTime, default=datetime.now)

    def __repr__(self):
        return f"<ShelfBook shelf_id={self.shelf_id} added_by={self.added_by}>"
```

`ShelfMembership` denotes who has membership to what shelf. It stores the shelf id, username, the wrapped key which is the shelf's own independent AES-256-GCM key, wrapped in their own RSA-2048 public key (as mentioned earlier).

`ShelfAccessRequest` class is for when a user searches the app for a shelf and asks to join. The user request to join, and the owner's username is stored in the database. Their key is wrapped and saved, with the shelf id saved along with it. The owner can view who wants to join their shelf in a separate tab and accept or reject as needed.

`ShelfBooks` are also a class; they have their work ID, shelf ID, and who it is added by. It stores a SHA-256 hash of the plaintext work ID with the encrypted work id encoded. This lets non-members check whether a specific book has reviews on a shelf by hashing a known work ID and comparing, without revealing the shelf's full book list.

```
def update_shelf_keys(shelf_id: int, wrapped_keys: dict, version: int):
    with SessionLocal() as session:
        for username, wrapped_key in wrapped_keys.items():
            m = session.query(ShelfMembership).filter_by(
                shelf_id=shelf_id, username=username
            ).first()
            if m:
                m.wrapped_key = wrapped_key
                m.key_version = version
                m.updated_at = datetime.now()
        session.commit()

def get_shelf_member_certificates(shelf_id: int) -> list[tuple[str, bytes]]:
    with SessionLocal() as session:
        members = session.query(ShelfMembership).filter_by(shelf_id=shelf_id).all()
        result = []
        for m in members:
            user = session.query(User).filter_by(username=m.username).first()
            if user:
```

```
                result.append((m.username, bytes(user.certificate)))
        return result
```

`get_shelf_member_certificates` fetches the remaining member's certificates. The certificates are used to extract each member's RSA public key and wrap the new AES-256 shelf key for them individually.

`update_shelf_keys` is the commit step. It receives the result of the wrapping and writes each new wrapped key back to `ShelfMembership` and increments the `key_version` .

```
def create_access_request(
    shelf_id: int, target_username: str, request_type: str, wrapped_key: bytes | None = None
) -> ShelfAccessRequest:
    with SessionLocal() as session:
        req = ShelfAccessRequest(
            shelf_id=shelf_id,
            target_username=target_username,
            request_type=request_type,
            wrapped_key=wrapped_key,
        )
        session.add(req)
        session.commit()
        session.refresh(req)
        session.expunge(req)
        return req
```

`create_access_request` is for when a user wants to join a shelf. We create a shelf request object with the relevant data and add it to the base.

Additional code

There is additional code for the frontend, API, and database, however this will not be documented as it was not relevant to the assignment.